

Part 13: Smoothing Relationships

Smoothing refers generally to procedures that attempt to extract underlying **signal** from **noise** in a **nonparametric** manner.

The term “nonparametric” means the use of a statistical model that does not make restrictive assumptions. The data are allowed to “speak for themselves” as much as possible.

Smoothing techniques are very commonly applied to situations where one variable is naturally thought of as a **response**, i.e., a quantity to be predicted, and other variables are considered as **predictors**, i.e., the **regression** problem.

Example: Pricing Options

A **European call option** on a stock is a contract that gives the holder the right to purchase that stock at the named **strike price** on the **expiration date**.

A **American call option** is the same, but allows the holder to purchase the stock at that price **on or before** the expiration date.

The classic **Black-Scholes Theory** for option pricing establishes a price for a European option as a function of the following: the strike price, the time to expiration, the current asset price, the volatility of the price, and the current risk-free rate of return.

We will consider a data set consisting of 1,000 call options sampled on September 29, 2017.

To generate this sample, NYSE stock symbols were randomly chosen, and then one currently-listed call option was randomly chosen for each equity, weighted by the volume of trading.

Information recorded for each were as follows: The strike price, the time to expiration, the current asset price, the historical volatility (based on daily returns of 30 most recent trading days), the **implied volatility** and the Black-Scholes price for the option (with a risk-free rate of zero assumed).

This sample can be found in the Data Sets area on Canvas, with the name optionssample09302017.txt.

```
In [1]: import pandas as pd

optionsdata = \
    pd.read_csv("optionssample09302017.txt", sep=",")
```

Our objective is to explore the relationships between the current ask price for the option and the available variables.

This is an “empirical” alternative to Black-Scholes theory, whose limitations are well-known, mainly due to the failure of the underlying normality assumption.

A classic manifestation of the failure of Black-Scholes is the so-called **volatility smile**. Under the theory, a plot of implied volatility versus how far “in the money” the option currently is should be flat for options with the same expiration date.

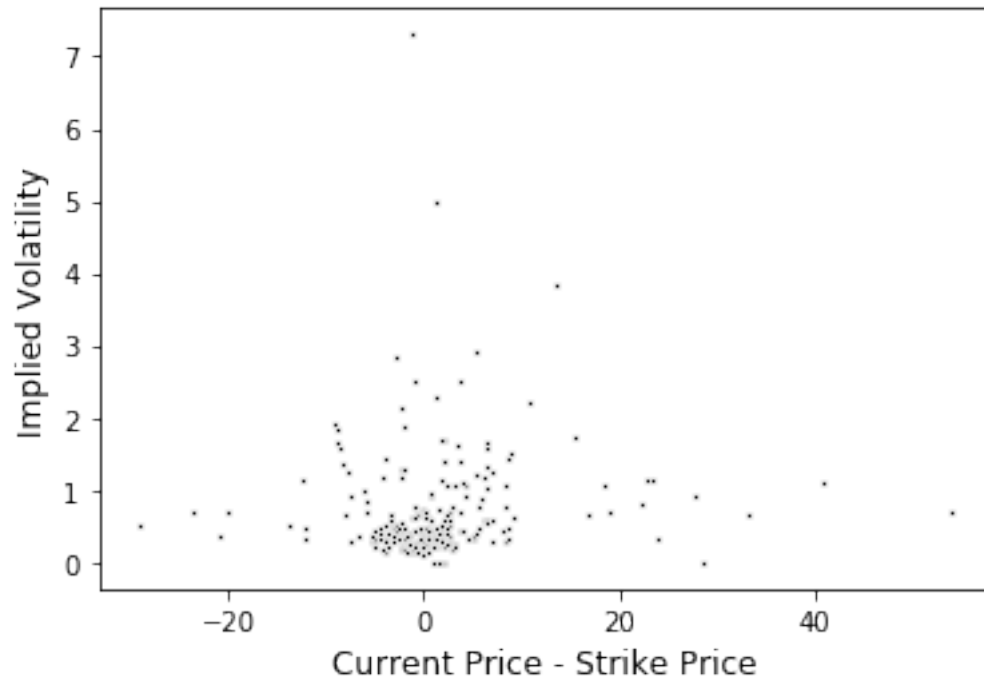
Let’s create this plot in our case. Is there evidence of the smile here?

```
In [2]: optionsdata["inmoney"] = \
        optionsdata["curprice"] - optionsdata["strike"]

        optionsdata20 = \
            optionsdata[optionsdata["timetoexpiry"] == 20]

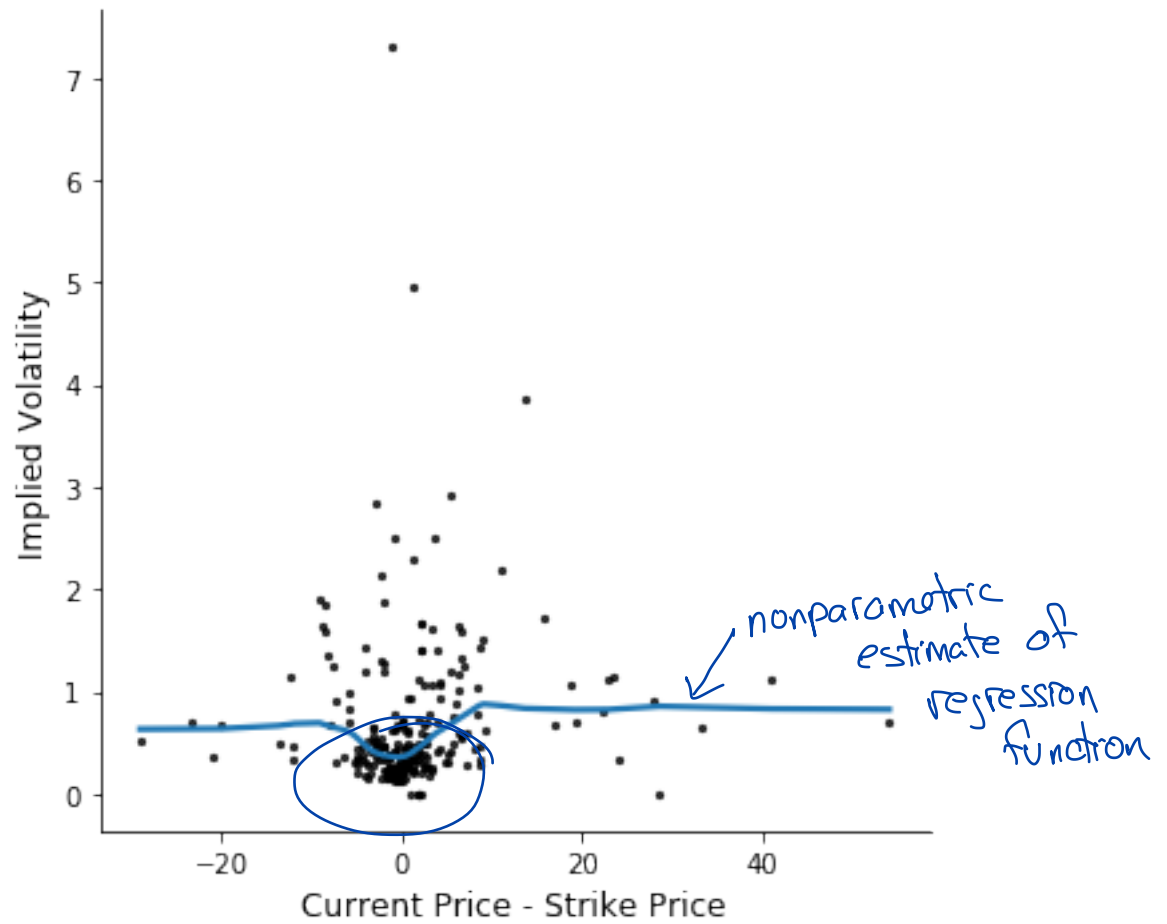
import seaborn as sns
import matplotlib.pyplot as plt

ax = sns.scatterplot(x="inmoney", y="implvol",
                    data=optionsdata20, s=5, color="black")
plt.xlabel("Current Price - Strike Price", size=12)
plt.ylabel("Implied Volatility", size=12)
plt.show()
```



The seaborn function `lmplot()` allows for the inclusion of a smoothed version of the relationship between the variables. This can be done by setting `lowess = True`.

```
In [3]: sns.lmplot(x="inmoney", y="implvol",  
                  data=optionsdata20, lowess=True,  
                  scatter_kws={"s": 5, "color": "black"})  
plt.xlabel("Current Price - Strike Price", size=12)  
plt.ylabel("Implied Volatility", size=12)  
plt.show()
```



Exercise: Comment on the addition of the smooth to the plot.

The smoother serves to highlight the underlying relationship.

Local Linear Regression

The figure on the previous page shows a **nonparametric regression** or **nonparametric smooth** of the scatter plot.

There are many variants of nonparametric regression, but here we focus on **local linear regression**, a version of local polynomial regression. This approach has proven to be very powerful, and possesses many strong theoretical properties to justify its use.

Briefly stated, this procedure works by fitting a sequence of linear models: Each is fit not to the entire data set, but to only data within a **neighborhood** of a target point. The size of this neighborhood is a **smoothing parameter**: Large neighborhoods yield a large degree of smoothing, while small neighborhoods result in minimal smoothing.

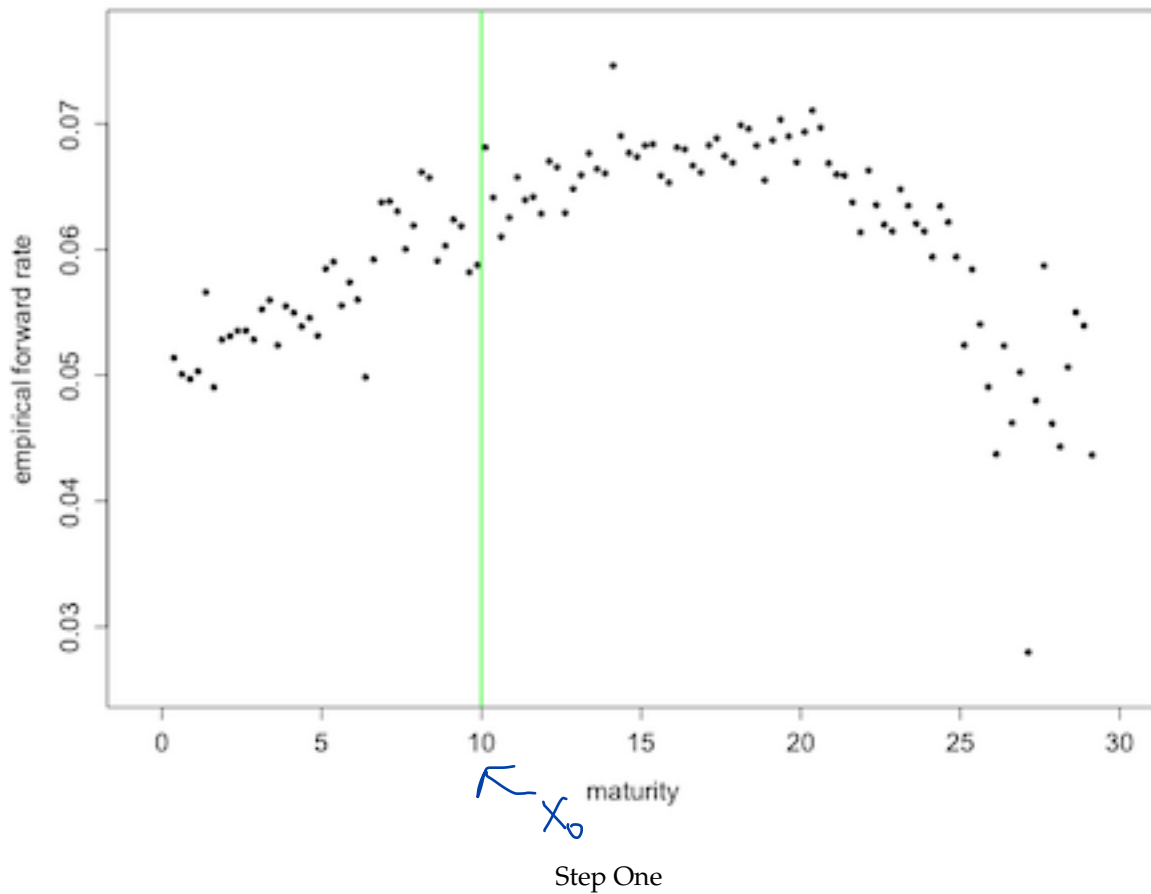
Our model here is that we observe (x_i, Y_i) for $i = 1, 2, \dots, n$ and that

$$Y_i = \underbrace{f(x_i)} + \underbrace{\epsilon_i}$$

where the ϵ_i are iid with mean zero and variance σ^2 . Assuming that the ϵ_i are normal will lead to further nice properties, but this development does not require that assumption.

In order to construct the local linear regression estimate of $f(\cdot)$, it is best to consider a sequence of steps for each fixed x_0 at which $f(\cdot)$ will be estimated.

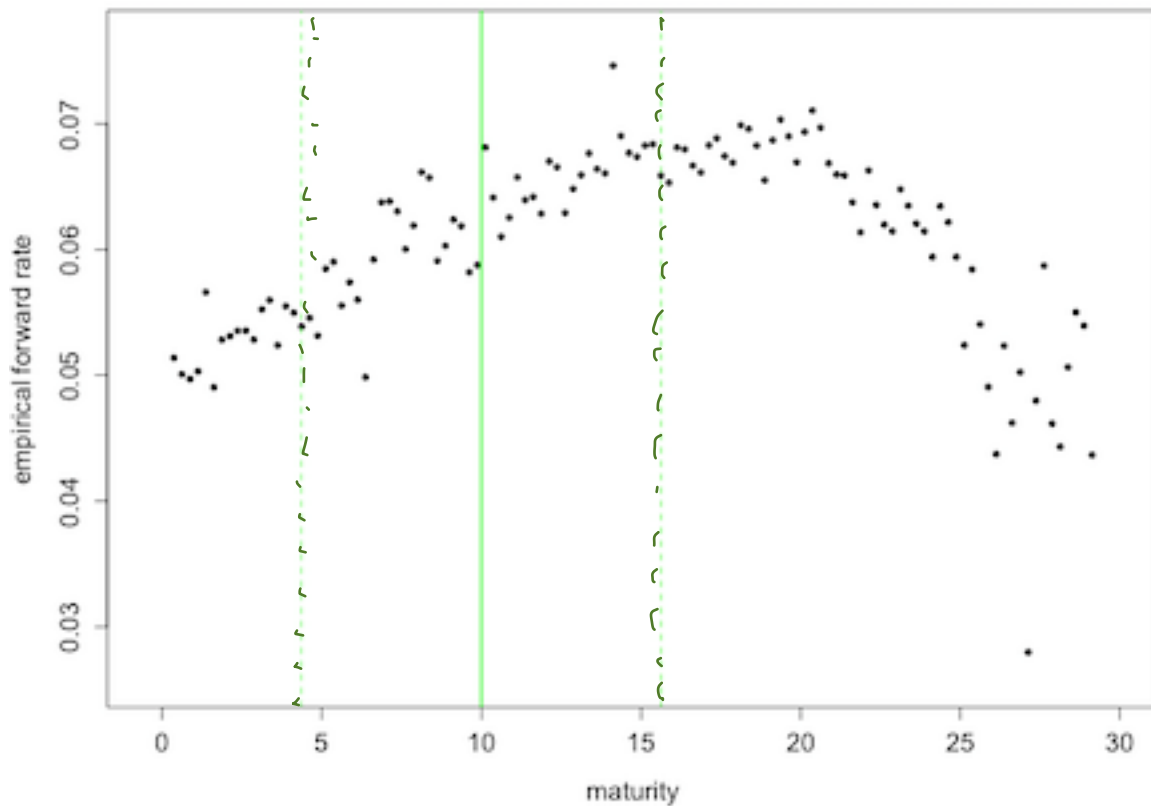
↑
target point (not necessarily
a data point)



Step One: Fix the target point x_0 .

Our objective is to estimate the regression function at x_0 .

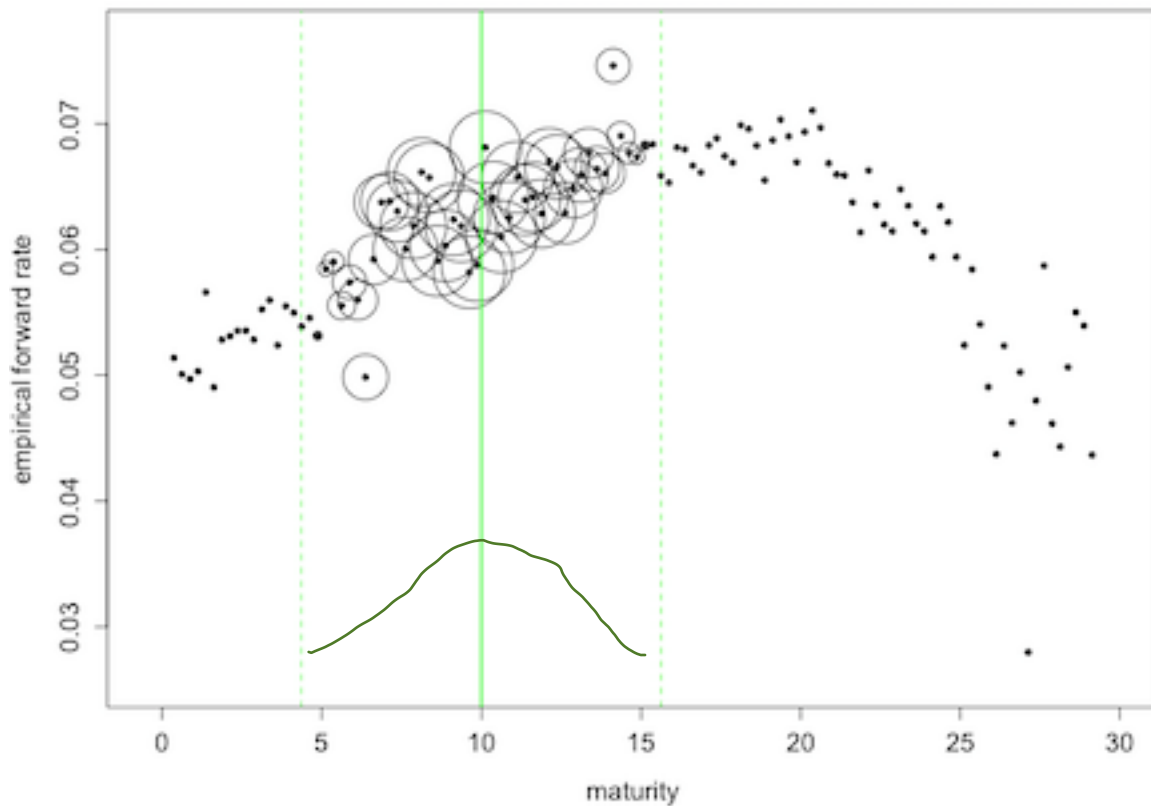
$$f(10)$$



Step Two

Step Two: Create the neighborhood around x_0 .

A common way to choose the neighborhood size is to choose is large enough to capture proportion α of the data. This parameter α is often called the **span**. A typical choice is $\alpha \approx 0.5$.

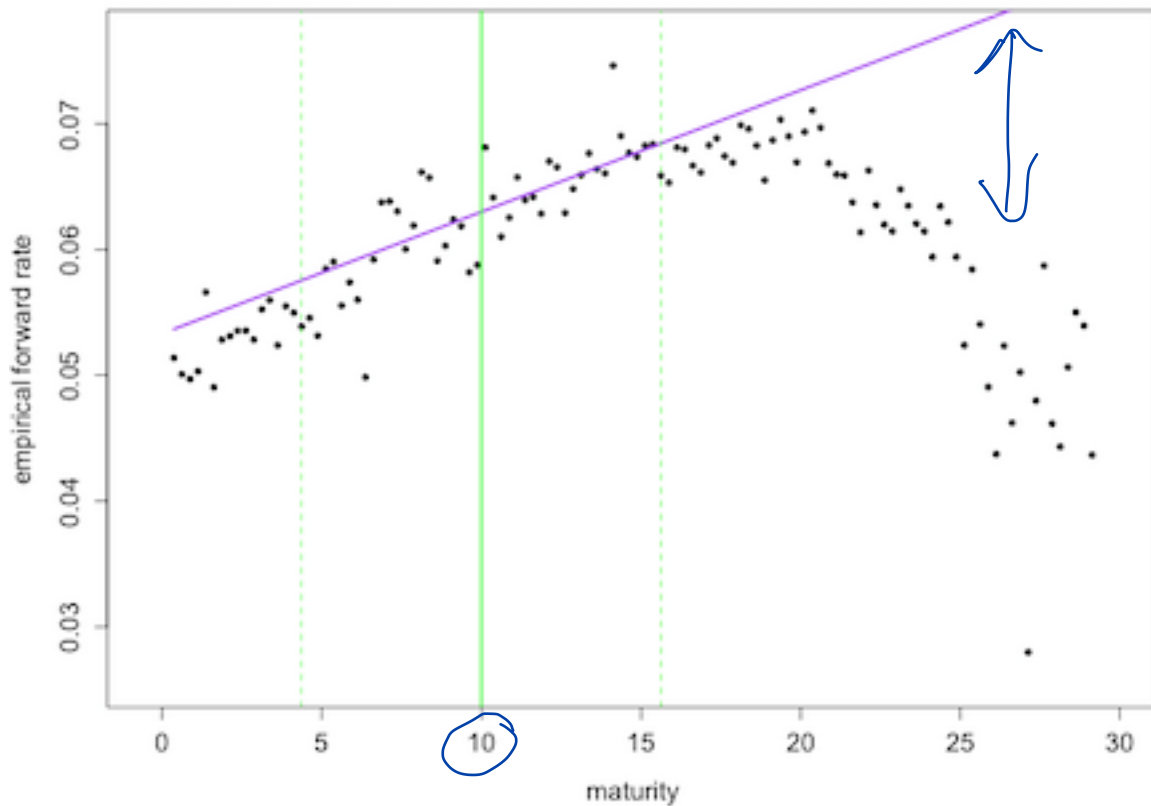


Step Three

Step Three: Weight the data in the neighborhood.

Values of x which are close x_0 will receive a larger weight than those far from x_0 . Denote by w_i the weight placed on observation i . A typical default choice is the **tri-cube weight function**:

$$w_i = \begin{cases} \left(1 - \left|\frac{x_i - x_0}{\max \text{ dist}}\right|^3\right)^3, & \text{if } x_i \text{ in the neighborhood of } x_0 \\ 0, & \text{if } x_i \text{ is not in neighborhood of } x_0 \end{cases}$$



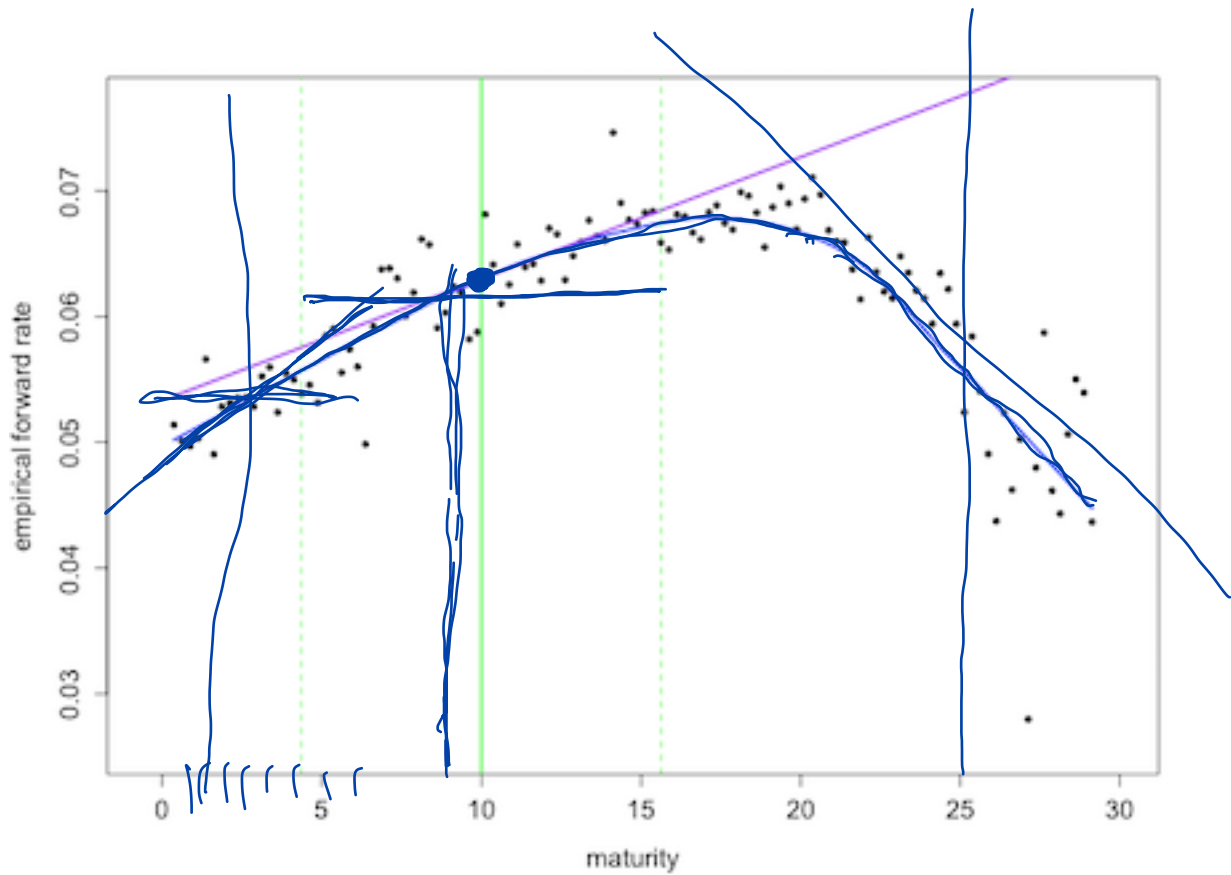
Step Four

Step Four: Fit the local regression line.

This is done by finding β_0 and β_1 to minimize the weighted sum of squares

$$\sum_{i=1}^n w_i (y_i - (\beta_0 + \beta_1 x_i))^2$$

weighted
least squares



Step Five

Step Five: Estimate $f(x_0)$.

This is done using the fitted regression line to estimate the regression function at x_0 :

$$\hat{f}(x_0) = \hat{\beta}_0 + \hat{\beta}_1 x_0$$

This is implemented in Python using `lowess()` in `statsmodels`. Note that the argument `frac` controls the span. The output of `lowess()` is a two-column matrix, where the first column is the sorted predictor values in the training set, and the second column is the fitted value for each.

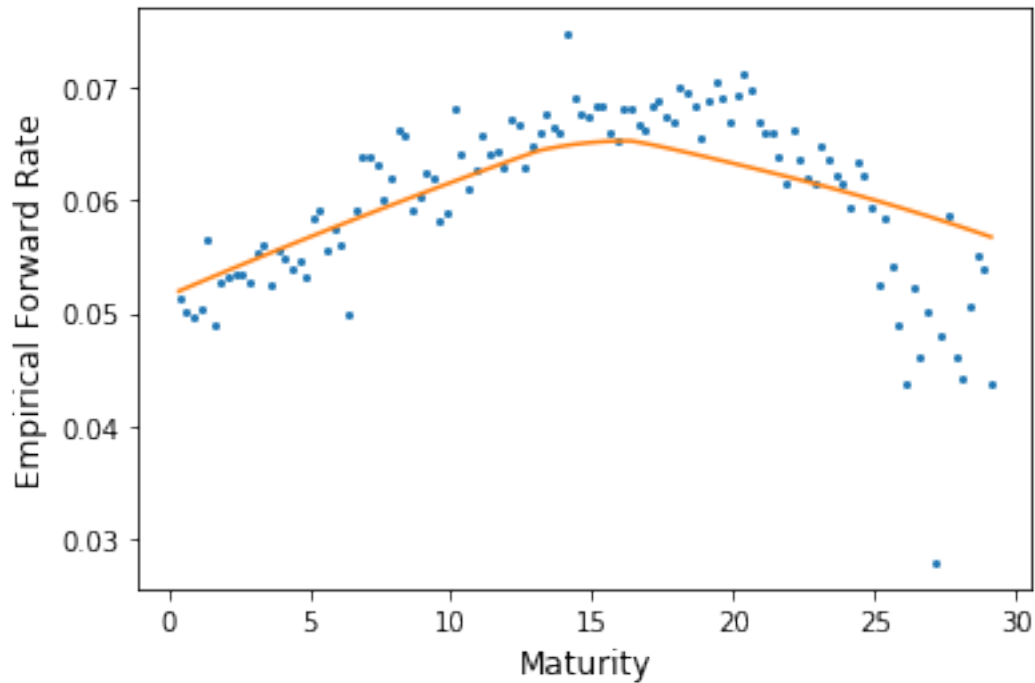
```
In [4]: import pandas as pd

forratedat = pd.read_csv(
    "http://www.stat.cmu.edu/~cschafer/MSCF/forratedat.txt",
    sep=" ")

import statsmodels.api as sm

lowout = sm.nonparametric.lowess(forratedat['emp'],
                                forratedat['maturity'], frac=0.90)

fig, axs = plt.subplots(1, 1, figsize=[6,4])
axs.plot(forratedat['maturity'], forratedat['emp'],
        marker='.', ms=4, linestyle="none")
axs.set_xlabel("Maturity", size=12)
axs.set_ylabel("Empirical Forward Rate", size=12)
axs.plot(lowout[:,0], lowout[:,1])
plt.show()
```



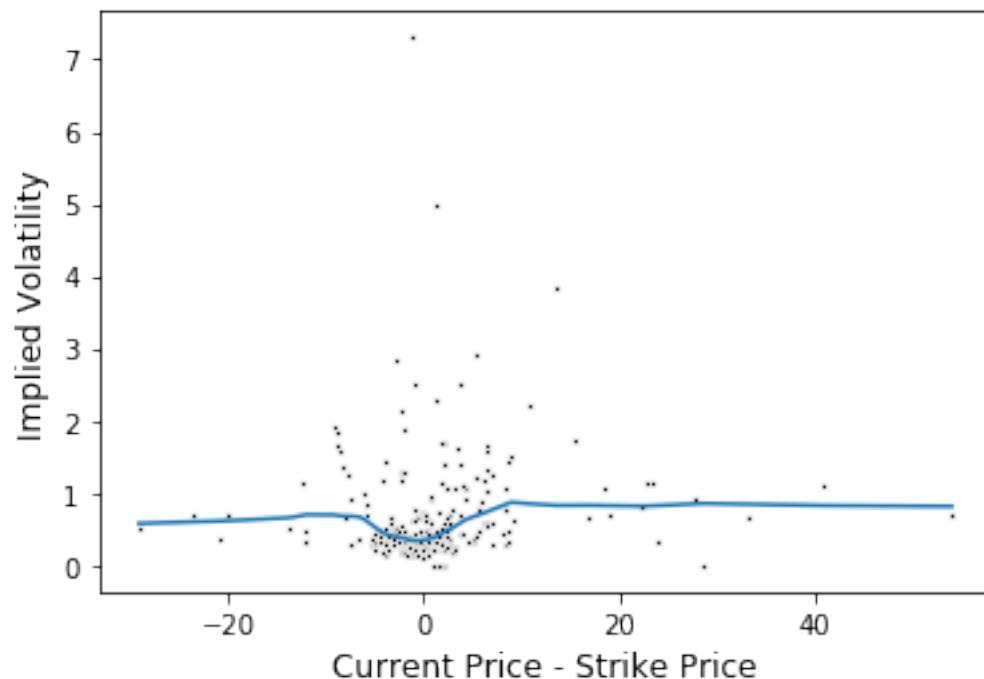
Exercise: Try refitting the model varying the span. Describe the effect.

Choosing span (frac) too large oversmooths,
while choosing too small leads to
non-smooth estimate

Let's also revisit the options example from above.

```
In [5]: lowopt = \
        sm.nonparametric.lowess(optionsdata20['implyvol'],
                                optionsdata20['inmoney'], frac=0.5)

ax = sns.scatterplot(x="inmoney", y="implyvol",
                    data=optionsdata20, s=5, color="black")
plt.xlabel("Current Price - Strike Price", size=12)
plt.ylabel("Implied Volatility", size=12)
plt.plot(lowopt[:,0], lowopt[:,1])
plt.show()
```



Smoothing parameter selection in Regression

We saw above that the choice of the span (the smoothing parameter) controls how “smooth” the resulting estimate is: A smaller value leads to a “rougher” estimate, a larger value gives a “smoother” estimate.

Exercise: Explain how the choice of the smoothing parameter relates to the **bias/variance tradeoff**.

If smoothing parameter is too small,
estimator will have high variance,
but have low bias

If chosen too large, have low
variance, but high bias

$$\text{MSE}(\hat{f}) = \text{bias}^2(\hat{f}) + V(\hat{f})$$

Automated approaches to smoothing parameter selection do exist. Different ideas are available, but a very popular approach is based on **cross validation**.

As we saw before, the motivation behind cross validation is that we seek a regression function that makes accurate predictions for **new** observations, not for those that are used to fit the model. A regression function that makes accurate predictions only for training data is said to be **overfit**.

Exercise: Discuss how nonparametric regression, and the choice of the smoothing parameter, leads to increased risk of overfitting. Does choosing the smoothing parameter to minimize residual sum of squares seem like a good idea?

NO. Nonparametric regression models are typically quite flexible, i.e. as vary smoothing parameter, get wide range of fits including models with very small RSS.

Hence, nonparametric approaches increase risk of overfitting.

The **residual sum of squares** in this case is

$$\text{RSS} = \sum_{i=1}^n \left(Y_i - \hat{f}_h(x_i) \right)^2$$

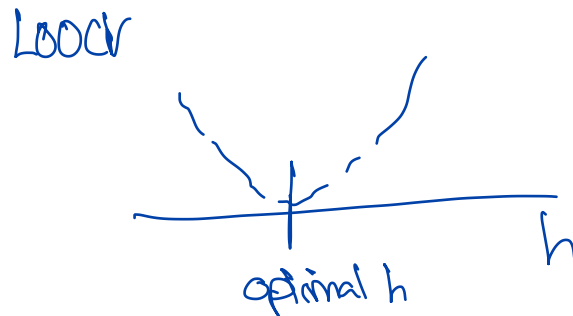
where $\hat{f}_h(x_i)$ is the estimate of the regression function $f()$ when smoothing parameter h is used.

Instead of choosing h to minimize RSS, we minimize the **leave-one-out cross validation score**:

$$\text{LOOCV} = \sum_{i=1}^n \left(Y_i - \hat{f}_h^{(-i)}(x_i) \right)^2$$

where $\hat{f}_h^{(-i)}(x_i)$ is the estimate of the regression function **when the i^{th} observation is excluded from the training set**.

Again, the idea here is that, by leaving out observation i , we are treating it as if it were a “new” observation, and testing to see how well this choice of h does in predicting the response.



Exercise: We note that for local linear regression, it is possible to write out a matrix $\mathbf{M}$ such that $\hat{\mathbf{Y}} = \mathbf{M}\mathbf{Y}$. What are the implications for the calculation of the leave-one-out cross validation score, and for the calculation of AIC?

$$\hat{\mathbf{Y}} = \mathbf{M} \mathbf{Y}$$

This allows us to use the shortcut formula, presented in discussion of PRESS, for calculating the leave-one-out residuals

$\text{tr}(\mathbf{M})$ gives a measure of the complexity of the model, "effective degrees of freedom", "equivalent dof"

hat matrix $\text{tr}(\mathbf{H}) = p+1$ because $\mathbf{H}$ is a projection matrix, its trace equals the dimension of space projected onto

Recall: $\text{AIC} = -2 \log \text{likelihood} + 2(\# \text{ of pars})$
 effective dof

There is an R function `loess.as()`, which is part of the package `fANCOVA`, which has automatic selection of the span.

```
In [6]: import rpy2
        %load_ext rpy2.ipython
```

Note the syntax for `loess.as()`: The predictor vector is given first, then the response. The criterion can be either `gcv` or `aicc`. “GCV” stands for **generalized cross validation**, which is an approximation to leave-one-out cross-validation. “AICC” is a “corrected” version of AIC.

The component `holdlo2$pars$span` holds the optimal span.

```
In [12]: %%R -i forratedat -o optspan

        library(fANCOVA)
        holdlo = loess.as(forratedat$maturity,
                           forratedat$emp, criterion = "aicc")
        optspan = holdlo$pars$span
```

```
In [17]: optspan[0]
```

```
Out[17]: 0.7038925300366445
```

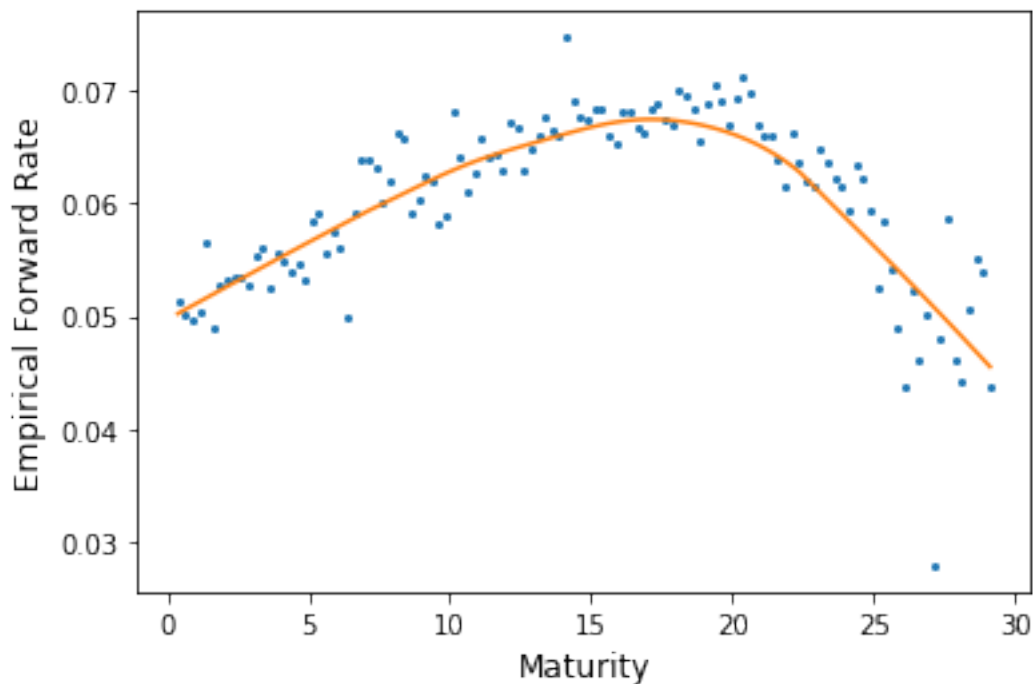
$$GCV: \hat{E}_{(-i)} = \frac{\hat{E}_i}{1 - h_{ii}}$$

with GCV, replace h_{ii} with $\text{tr}(H)/n = \bar{h}$

Let's remake the plot.

```
In [14]: lowout = sm.nonparametric.lowess(forratedat['emp'],
                                           forratedat['maturity'], frac=optspan[0])

fig, axs = plt.subplots(1, 1, figsize=[6,4])
axs.plot(forratedat['maturity'], forratedat['emp'],
        marker='.', ms=4, linestyle="none")
axs.set_xlabel("Maturity", size=12)
axs.set_ylabel("Empirical Forward Rate", size=12)
axs.plot(lowout[:,0], lowout[:,1])
plt.show()
```



Exercise: Comment on the quality of this fit. Use $\text{criterion} = \overset{\text{gcv}}{\text{aic}}$ instead. How does the performance change? What do you think happened?

When using GCV, the span is much smaller, and the estimate is less smooth.

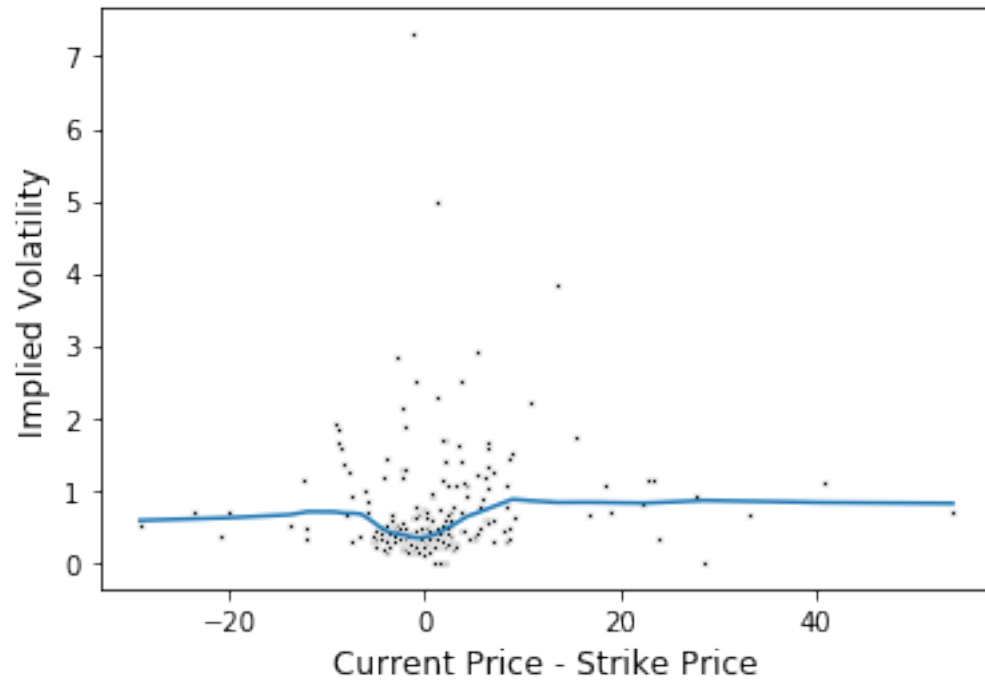
In general, GCV (CV) is less reliable with small samples, and AIC is more stable.

Let's now reconsider the options example.

```
In [15]: %%R -i optionsdata20 -o optspan
```

```
holdlo = loess.as(optionsdata20$inmoney,  
                  optionsdata20$implvol, criterion = "gcv")  
optspan = holdlo$pars$span
```

```
In [16]: lowopt2 = \  
        sm.nonparametric.lowess(optionsdata20['implvol'],  
                                optionsdata20['inmoney'], frac=optspan[0])  
  
ax = sns.scatterplot(x="inmoney", y="implvol",  
                    data=optionsdata20, s=5, color="black")  
plt.xlabel("Current Price - Strike Price", size=12)  
plt.ylabel("Implied Volatility", size=12)  
plt.plot(lowopt[:,0], lowopt[:,1])  
plt.show()
```



1 Part 14: Curse of Dimensionality

2 Summary

3 Often data come to us in a **high-dimensional form**, meaning that each ob-
4 servation is actually best thought of as a vector of many numbers.

5 In some situations, it is crucial that lower-dimensional representations of
6 these data are obtained. Here we present some approaches to doing that.

In Part 15

7 First, we motivate why this dimension reduction can be so important.

1 **The Curse of Dimensionality**

- 2 Despite the promise of nonparametric smoothing approaches, they suffer
3 from a serious drawback: The amount of data required to fit nonparamet-
4 ric regressions well increases exponentially with the dimension of the data.
5 This is called the **Curse of Dimensionality**.

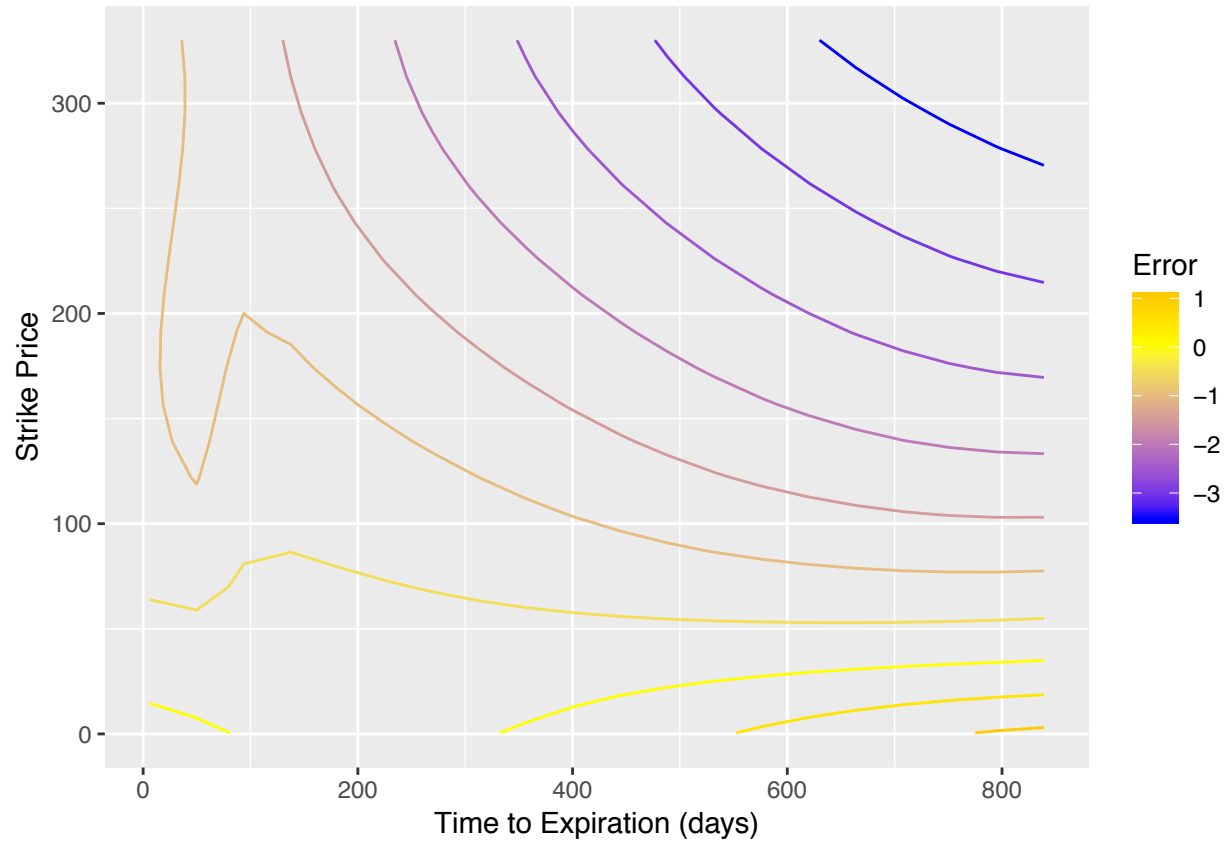
- 1 To start, we ask: **What is the “dimension” of data?**
- 2 Standard data come to us in the form of numbers, but a single **observation**
- 3 is often best thought of as a list of numbers.
- 4 For example, a **yield curve** shows how bond rates vary as a function of
- 5 time to maturity. A single yield curve consists of several numbers, usually
- 6 around ten rates. This count of how many numbers it takes to represent a
- 7 yield curve is called its **dimension**. We would say that a single yield curve
- 8 is “ten-dimensional.”
- 9 Another example: Imagine our data consists of separate price time series
- 10 for each of 100 stocks. Each series goes back 20 trading days. We would say
- 11 in this case that we have “100 observations of 20-dimensional time series”.

1 The dimension of data often comes up is in the context of regression or
2 supervised learning. If there are p predictors being used in a model for a
3 response variable, we would say that we are using a “ p -dimensional pre-
4 dictor vector.”

5 Nonparametric regression is possible when the predictor vector is p -
6 dimensional. For instance, if $p = 2$, a neighborhood can be thought of
7 as a square centered on the target value. Local linear regression involves
8 fitting a plane within this square.

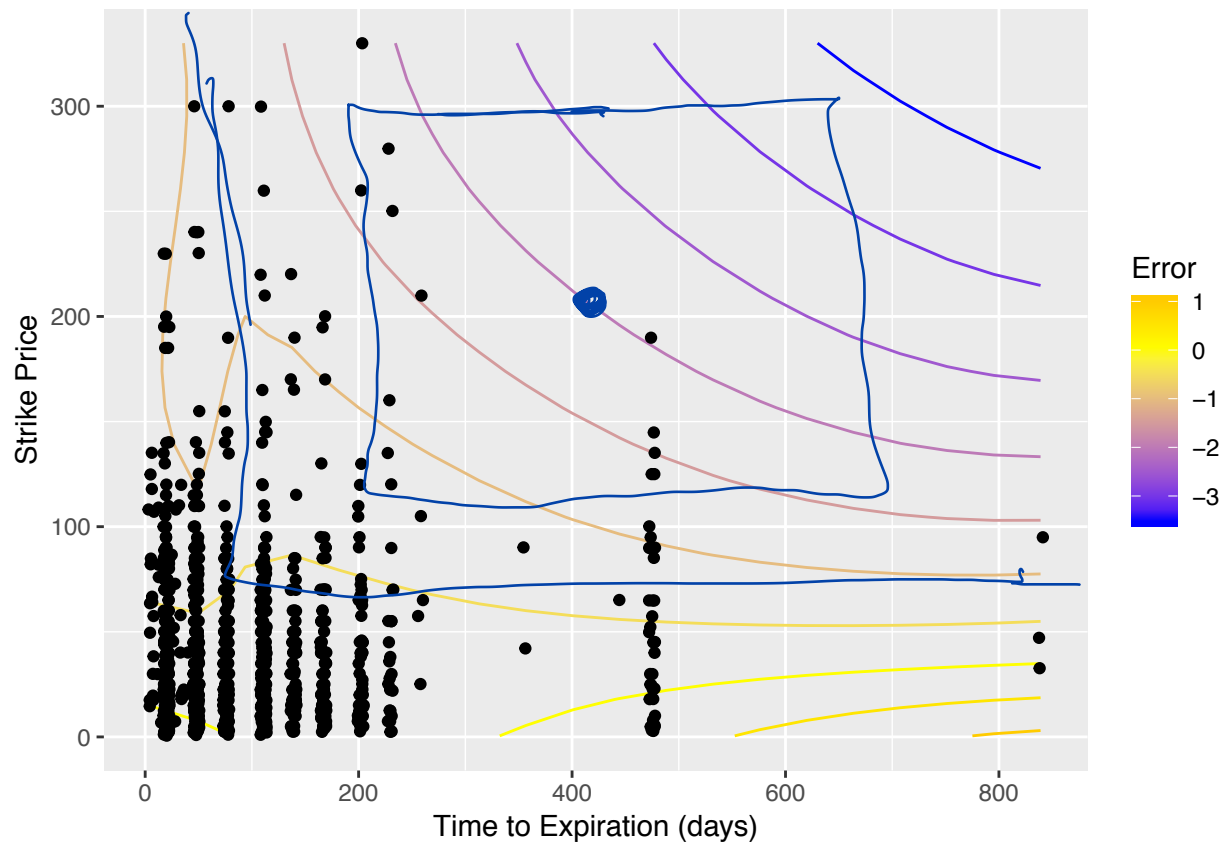


9 Reconsider our options sample from the previous Part. The figure on the
10 following page shows a two-dimensional local linear regression fit. The
11 response is the “error” in the Black-Scholes price (relative to the ask price)
12 and the predictors are the time to expiration and the strike price.



Exercise: The next plot adds on the 1,000 observed predictor pairs used in this case. Sketch the neighborhood when the regression function is estimated at $(400, 200)$. What will have to happen in order for this estimate to have sufficiently low variance?

In order for the estimator to have sufficiently low variance, choose nbhd larger than I would have hoped.



- 1 This example shows the fundamental challenge of working with high-
- 2 dimensional predictors: As the dimension increases, the available obser-
- 3 vations become more “spread out.” Neighborhoods must be made larger
- 4 in order to compensate and achieve estimates with acceptable variance.
- 5 But, choosing a large neighborhood takes away a key feature of using non-
- 6 parametric approaches: We want the estimate to be flexible, and not be
- 7 based on too much “smoothing.”

- 1 Consider the plot on the following slide.
- 2 Here, the sample size is 100. Each observation is two-dimensional, i.e.,
- 3 imagine these are the two predictors. These data are simulated from uni-
- 4 form distributions.
- 5 The “marks” shown in each margin depict the **marginal distribution** for
- 6 each predictor. Note that in the margins, the data appear to be quite closely
- 7 spaced. If we were to fit a nonparametric regression using just one predic-
- 8 tor, neighborhoods could be chosen small.
- 9 But, in two dimensions, large gaps appear in the distribution of the obser-
- 10 vations. Neighborhoods will have to be chosen larger in order to capture
- 11 enough data.

